SHARD.MEMORY — Blueprint Architetturale

Il Progetto

Replicare l'intera architettura di Supermemory come modulo self-hosted per SHARD. Stack: **Python** | DB: **SQLite + ChromaDB** | Embedding: **OpenAI/Groq API** | LLM extraction: **Groq (Llama 3)**

Architettura a 5 Layer

```
┌──────────────────────────────────────────────────────────────┐
│                        SHARD.MEMORY                          │
│                                                              │
│   ┌──────────┐   ┌──────────┐   ┌──────────┐   ┌──────────┐ │
│   │ INGEST   │→  │ EXTRACT  │→  │ GRAPH    │→  │ QUERY    │ │
│   │ Layer    │   │ Layer    │   │ Layer    │   │ Layer    │ │
│   └──────────┘   └──────────┘   └──────────┘   └──────────┘ │
│        ↑                              ↕                      │
│   ┌──────────┐                 ┌──────────────┐             │
│   │CONNECTOR │                 │   PROFILE    │             │
│   │ Layer    │                 │   Engine     │             │
│   └──────────┘                 └──────────────┘             │
│                                                              │
│  Storage: SQLite (facts/relations) + ChromaDB (vectors)      │
└──────────────────────────────────────────────────────────────┘
```

Layer 1: INGEST — Ingestion & Content Processing

Responsabilità

Accetta qualsiasi tipo di contenuto, lo normalizza in testo pulito, e lo passa al layer di estrazione.

Content Types Supportati

Tipo	Strategia di Estrazione	Libreria
Testo raw	Passthrough	-
Conversazioni	Parse strutturato (speaker + content)	-
PDF	Estrazione testo + OCR fallback	`pymupdf` + `pytesseract`
Immagini	OCR	`pytesseract` + `Pillow`

Tipo	Strategia di Estrazione	Libreria
Codice	AST-aware chunking	`tree-sitter`
Markdown	Chunking per heading hierarchy	`markdown-it-py`
HTML/URL	Fetch + clean + chunk	`trafilatura`
Video	Transcription audio	`whisper` (local small model)

Chunking Strategies

```python
class ChunkStrategy(Enum):
    SEMANTIC = "semantic"       # Per documenti: split per sezioni/paragrafi
    AST_AWARE = "ast_aware"     # Per codice: rispetta confini funzione/classe
    HEADING = "heading"         # Per markdown: split per heading level
    SLIDING = "sliding"         # Fallback: sliding window con overlap
    CONVERSATION = "conversation"  # Per chat: per turno/messaggio

class Chunk:
    id: str                     # UUID
    content: str                # Testo del chunk
    source_doc_id: str          # ID documento sorgente
    chunk_index: int            # Posizione nel documento
    metadata: dict              # Tipo, timestamp, speaker, etc.
    embedding: list[float]      # Vector embedding (generato dopo)
```

Pipeline di Processing

```
Input → Detect Type → Extract Text → Clean → Chunk → Embed → Store
                                                ↓
                                        Memory Extraction
                                        (Layer 2)
```

Layer 2: EXTRACT — Memory Extraction Engine

Il Cuore del Sistema

Questo è il pezzo che differenzia un memory system da un semplice RAG. Ogni chunk passa attraverso un LLM che estrae **fatti strutturati**.

Prompt di Estrazione (via Groq/Llama 3)

```python
EXTRACTION_PROMPT = """
Analyze the following content and extract discrete facts.
For each fact, classify it:

TYPES:
- FACT: Stable information (e.g., "Andrea lives in Zevio")
- PREFERENCE: User preference (e.g., "Prefers Python over JS")
- EPISODE: Time-bound event (e.g., "Had meeting with investor today")
- GOAL: Active objective (e.g., "Building SHARD autonomous scaffold")
- RELATION: Relationship between entities (e.g., "Joshua plays guitar in Coalesce Di

For each fact, also determine:
- TEMPORAL: Is this time-sensitive? When does it expire?
- CONFIDENCE: How confident are you this is a real fact? (0.0-1.0)
- ENTITIES: What named entities are involved?

Return JSON array of extracted memories.

Content:
{content}

Container context: {container_tag}
"""
```

Struttura Memory

```python
```

```python
@dataclass
class Memory:
    id: str                         # UUID
    content: str                    # Il fatto estratto in linguaggio naturale
    memory_type: MemoryType         # FACT | PREFERENCE | EPISODE | GOAL | RELATION
    entities: list[str]             # Entità coinvolte
    confidence: float               # 0.0 - 1.0

    # Temporal
    created_at: datetime
    expires_at: datetime | None  # None = non scade
    is_latest: bool                 # Per contradiction resolution

    # Relazioni nel grafo
    updates: str | None             # ID della memory che questa aggiorna
    extends: str | None             # ID della memory che questa estende
    derived_from: list[str]         # IDs delle memories da cui è derivata

    # Source tracking
    source_doc_id: str
    source_chunk_id: str
    container_tag: str              # Scoping (progetto, utente, etc.)

    # Vector
    embedding: list[float]
```

Contradiction Resolution

```python
```

```python
async def resolve_contradictions(new_memory: Memory, existing_memories: list[Memory]
    """
    Quando un nuovo fatto contraddice uno esistente:
    1. Cerca memories simili (cosine similarity > 0.85)
    2. Chiedi all'LLM se c'è contraddizione
    3. Se sì: marca la vecchia come is_latest=False,
       la nuova come updates=old_id
    """

    similar = await vector_search(new_memory.embedding, threshold=0.85)

    for existing in similar:
        contradiction = await llm_check_contradiction(
            existing.content,
            new_memory.content
        )

        if contradiction.is_contradicting:
            existing.is_latest = False
            new_memory.updates = existing.id
            new_memory.is_latest = True

            # Log per audit
            log_contradiction(existing, new_memory, contradiction.reason)
```

Derivation Engine

```python
```

```python
async def derive_insights(memories: list[Memory], container_tag: str):
    """
    Periodicamente (o dopo N nuove memories), analizza il grafo
    per derivare nuovi fatti impliciti.

    Es: "Andrea è vocalist di Coalesce Divide" +
        "Andrea fa marketing per LocalFlow" →
        Derivato: "Andrea ha competenze sia creative che business"
    """

    DERIVATION_PROMPT = """
    Given these known facts about {container_tag}:
    {facts_list}

    Can you infer any NEW facts that aren't explicitly stated
    but are strongly implied? Only high-confidence inferences.

    Return JSON array of derived facts with source_ids.
    """

    derived = await llm_derive(memories)
    for d in derived:
        d.derived_from = [m.id for m in d.source_memories]
        d.confidence *= 0.8  # Derivati hanno confidence ridotta
```

Layer 3: GRAPH — Knowledge Graph & Storage

Dual Storage Architecture

```
┌──────────────────────┐      ┌──────────────────────┐
│   SQLite (Facts)     │      │  ChromaDB (Vectors)  │
│                      │      │                      │
│ • memories table     │      │ • memory_embeddings  │
│ • relations table    │      │ • chunk_embeddings   │
│ • documents table    │      │                      │
│ • profiles table     │      │                      │
│ • containers table   │      │                      │
└──────────────────────┘      └──────────────────────┘

        ↕                            ↕
    Relational queries          Semantic search
    (exact match,               (similarity,
     temporal, graph             hybrid retrieval)
     traversal)
```

Schema SQLite

```sql
```

```sql
-- Documenti sorgente
CREATE TABLE documents (
    id TEXT PRIMARY KEY,
    content_type TEXT NOT NULL,      -- pdf, text, conversation, code, etc.
    title TEXT,
    source_url TEXT,
    container_tag TEXT NOT NULL,
    status TEXT DEFAULT 'queued',    -- queued→extracting→chunking→embedding→indexin
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    metadata JSON
);

-- Chunks dei documenti
CREATE TABLE chunks (
    id TEXT PRIMARY KEY,
    document_id TEXT REFERENCES documents(id),
    content TEXT NOT NULL,
    chunk_index INTEGER,
    chunk_strategy TEXT,
    metadata JSON,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Memories estratte (il cuore)
CREATE TABLE memories (
    id TEXT PRIMARY KEY,
    content TEXT NOT NULL,
    memory_type TEXT NOT NULL,       -- FACT, PREFERENCE, EPISODE, GOAL, RELATION
    entities JSON,                   -- ["Andrea", "SHARD", "Groq"]
    confidence REAL DEFAULT 1.0,

    is_latest BOOLEAN DEFAULT TRUE,
    expires_at TIMESTAMP,

    source_doc_id TEXT REFERENCES documents(id),
    source_chunk_id TEXT REFERENCES chunks(id),
    container_tag TEXT NOT NULL,

    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Relazioni tra memories (il grafo)
CREATE TABLE memory_relations (
    id TEXT PRIMARY KEY,
    source_id TEXT REFERENCES memories(id),
```

```sql
    target_id TEXT REFERENCES memories(id),
    relation_type TEXT NOT NULL,     -- UPDATES, EXTENDS, DERIVES
    metadata JSON,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Profili utente/container (auto-generati)
CREATE TABLE profiles (
    container_tag TEXT PRIMARY KEY,
    static_facts JSON,              -- ["Andrea vive a Zevio", "Vocalist metalcore"]
    dynamic_facts JSON,             -- ["Sta sviluppando SHARD.MEMORY", "Focus su pi
    last_updated TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Indici
CREATE INDEX idx_memories_container ON memories(container_tag);
CREATE INDEX idx_memories_type ON memories(memory_type);
CREATE INDEX idx_memories_latest ON memories(is_latest);
CREATE INDEX idx_memories_expires ON memories(expires_at);
CREATE INDEX idx_relations_source ON memory_relations(source_id);
CREATE INDEX idx_relations_target ON memory_relations(target_id);
```

ChromaDB Collections

```python
# Collection per le memories (semantic search)
memory_collection = chroma_client.get_or_create_collection(
    name="memories",
    metadata={"hnsw:space": "cosine"}
)

# Collection per i chunks raw (RAG)
chunk_collection = chroma_client.get_or_create_collection(
    name="chunks",
    metadata={"hnsw:space": "cosine"}
)
```

Automatic Forgetting

```python
```

```python
async def garbage_collect():
    """
    Esegui periodicamente (ogni ora o dopo ogni sessione).
    1. Rimuovi memories scadute (expires_at < now)
    2. Decadi confidence delle EPISODE memories vecchie
    3. Rimuovi memories con confidence < 0.1
    """

    now = datetime.utcnow()

    # 1. Expired memories
    expired = db.execute(
        "UPDATE memories SET is_latest = FALSE WHERE expires_at < ? AND is_latest =
        (now,)
    )

    # 2. Episode decay (half-life di 7 giorni)
    episodes = db.execute(
        "SELECT id, confidence, created_at FROM memories WHERE memory_type = 'EPISOD
    )
    for ep in episodes:
        age_days = (now - ep.created_at).days
        new_confidence = ep.confidence * (0.5 ** (age_days / 7))
        db.execute("UPDATE memories SET confidence = ? WHERE id = ?", (new_confidenc

    # 3. Cleanup sotto soglia
    db.execute("DELETE FROM memories WHERE confidence < 0.1 AND memory_type = 'EPISO
```

Layer 4: QUERY — Search & Retrieval

Tre Modalità di Search

```python
class SearchMode(Enum):
    HYBRID = "hybrid"       # RAG + Memory insieme (default)
    MEMORIES = "memories"   # Solo memories estratte
    DOCUMENTS = "documents" # Solo chunks RAG raw
```

Hybrid Search Pipeline

```python
```

```python
async def hybrid_search(
    query: str,
    container_tag: str,
    search_mode: SearchMode = SearchMode.HYBRID,
    top_k: int = 10,
    rerank: bool = False,
    rewrite_query: bool = False
) -> SearchResult:

    # 1. Optional: Query rewriting per espandere la query
    if rewrite_query:
        expanded = await llm_rewrite_query(query)
    else:
        expanded = query

    # 2. Genera embedding della query
    query_embedding = await get_embedding(expanded)

    # 3. Search basato sulla modalità
    results = []

    if search_mode in (SearchMode.HYBRID, SearchMode.MEMORIES):
        # Cerca nelle memories (solo is_latest=True)
        memory_results = memory_collection.query(
            query_embeddings=[query_embedding],
            n_results=top_k,
            where={
                "$and": [
                    {"container_tag": container_tag},
                    {"is_latest": True}
                ]
            }
        )
        results.extend(memory_results)

    if search_mode in (SearchMode.HYBRID, SearchMode.DOCUMENTS):
        # Cerca nei chunks RAG
        chunk_results = chunk_collection.query(
            query_embeddings=[query_embedding],
            n_results=top_k,
            where={"container_tag": container_tag}
        )
        results.extend(chunk_results)

    # 4. Optional: Reranking con cross-encoder
    if rerank:
```

```python
        results = await rerank_results(query, results)

    # 5. Deduplica e ordina
    return deduplicate_and_rank(results)
```

Query Rewriting

```python
async def llm_rewrite_query(query: str) -> str:
    """Espande query corte in query più ricche per migliorare recall."""

    prompt = f"""
    Rewrite this search query to be more comprehensive.
    Add related terms, synonyms, and context.
    Keep it as a search query, not a sentence.

    Original: "{query}"
    Rewritten:
    """
    return await llm_complete(prompt)
```

Reranking (opzionale, cross-encoder leggero)

```python
async def rerank_results(query: str, results: list, top_k: int = 5) -> list:
    """
    Usa l'LLM come reranker leggero.
    Per il Geekom A5, evita modelli cross-encoder pesanti.
    Usa Groq per speed.
    """

    prompt = f"""
    Query: "{query}"

    Rank these results by relevance (most relevant first):
    {json.dumps([r.content for r in results])}

    Return JSON array of indices in order of relevance.
    """

    ranking = await llm_complete(prompt)
    return [results[i] for i in ranking[:top_k]]
```

Layer 5: PROFILE — User Profile Engine

Auto-Generated Profiles

```python
```

```python
async def build_profile(container_tag: str) -> Profile:
    """
    Costruisce automaticamente un profilo dal grafo delle memories.
    ~50ms target (tutto locale tranne embedding).
    """

    # 1. Raccogli tutte le memories attive per questo container
    active_memories = db.execute("""
        SELECT content, memory_type, confidence, created_at
        FROM memories
        WHERE container_tag = ? AND is_latest = TRUE AND confidence > 0.3
        ORDER BY confidence DESC, created_at DESC
    """, (container_tag,))

    # 2. Separa static vs dynamic
    static_candidates = [m for m in active_memories
                         if m.memory_type in ('FACT', 'PREFERENCE', 'RELATION')
                         and m.confidence > 0.7]

    dynamic_candidates = [m for m in active_memories
                          if m.memory_type in ('EPISODE', 'GOAL')
                          or m.created_at > (datetime.utcnow() - timedelta(days=7))]

    # 3. Comprimi con LLM per leggibilità
    static_facts = await llm_summarize_facts(static_candidates, max_items=15)
    dynamic_facts = await llm_summarize_facts(dynamic_candidates, max_items=10)

    # 4. Cache nel DB
    profile = Profile(
        container_tag=container_tag,
        static_facts=static_facts,
        dynamic_facts=dynamic_facts,
        last_updated=datetime.utcnow()
    )

    db.execute("""
        INSERT OR REPLACE INTO profiles (container_tag, static_facts, dynamic_facts,
        VALUES (?, ?, ?, ?)
    """, (container_tag, json.dumps(static_facts), json.dumps(dynamic_facts), profil

    return profile
```

Profile + Search in One Call

```python
```

```python
async def profile_search(
    container_tag: str,
    query: str | None = None
) -> ProfileSearchResult:
    """
    L'equivalente di client.profile() di Supermemory.
    Ritorna profilo + risultati search opzionali in una chiamata.
    """

    # Profile (cached, rebuild se > 1 ora)
    profile = await get_or_build_profile(container_tag)

    # Optional search
    search_results = None
    if query:
        search_results = await hybrid_search(query, container_tag)

    return ProfileSearchResult(
        profile=profile,
        search_results=search_results
    )
```

CONNECTOR Layer — External Data Sync

Architettura Connettori

```python
```

```python
class BaseConnector(ABC):
    """Base class per tutti i connettori."""

    @abstractmethod
    async def sync(self, container_tag: str) -> list[Document]:
        """Sincronizza dati dall'esterno."""
        pass

    @abstractmethod
    async def watch(self, callback: Callable) -> None:
        """Webhook/polling per aggiornamenti real-time."""
        pass

class FileSystemConnector(BaseConnector):
    """Monitora una directory locale per nuovi file."""

    def __init__(self, watch_dir: str):
        self.watch_dir = watch_dir
        self.observer = Observer()  # watchdog

    async def sync(self, container_tag: str):
        files = glob.glob(f"{self.watch_dir}/**/*", recursive=True)
        return [await ingest_file(f, container_tag) for f in files]

class GitConnector(BaseConnector):
    """Sincronizza da un repo Git locale."""
    pass

class SessionLogConnector(BaseConnector):
    """Importa i log delle sessioni SSJ di SHARD."""
    pass
```

Connettori Prioritari per SHARD

Connettore	Priorità	Uso
SessionLog	P0	Importa log SSJ automaticamente
FileSystem	P0	Monitora directory SHARD per nuovi file
Git	P1	Tracka modifiche al codebase
Markdown/Notes	P1	Importa EMERGENT_BEHAVIORS.md etc.
Web/URL	P2	Salva pagine di documentazione

API Interna (per integrazione con SHARD)

```python
```

```python
class ShardMemory:
    """API principale — l'equivalente del client Supermemory."""

    def __init__(self, db_path: str = "shard_memory.db", chroma_path: str = "./chrom
        self.db = SQLiteDB(db_path)
        self.chroma = ChromaDB(chroma_path)
        self.extractor = MemoryExtractor()
        self.profiler = ProfileEngine()

    # === INGEST ===
    async def add(
        self,
        content: str,
        container_tag: str = "default",
        content_type: str = "text",
        metadata: dict = None
    ) -> Document:
        """Aggiungi contenuto. Estrae memories automaticamente."""
        pass

    async def add_conversation(
        self,
        messages: list[dict],  # [{"role": "user", "content": "..."}]
        container_tag: str = "default"
    ) -> Document:
        """Aggiungi una conversazione. Ottimizzato per chat LLM."""
        pass

    async def upload_file(
        self,
        file_path: str,
        container_tag: str = "default"
    ) -> Document:
        """Upload e processa un file."""
        pass

    # === SEARCH ===
    async def search(
        self,
        query: str,
        container_tag: str = "default",
        search_mode: str = "hybrid",
        top_k: int = 10,
        rerank: bool = False
    ) -> list[SearchResult]:
        """Cerca nel knowledge graph."""
```

```python
        pass

    # === PROFILE ===
    async def profile(
        self,
        container_tag: str = "default",
        query: str = None
    ) -> ProfileSearchResult:
        """Profilo + search opzionale in una chiamata."""
        pass

    # === MEMORY MANAGEMENT ===
    async def forget(
        self,
        memory_id: str = None,
        query: str = None,
        container_tag: str = None
    ) -> bool:
        """Dimentica una memory specifica o per query."""
        pass

    async def gc(self) -> GCResult:
        """Garbage collection: rimuovi expired, decadi episodi."""
        pass

    # === CONNECTORS ===
    def register_connector(self, connector: BaseConnector, container_tag: str):
        """Registra un connettore per sync automatico."""
        pass
```

Dipendenze Python

```txt
```

```
# Core
chromadb>=0.5.0          # Vector store locale
openai>=1.0.0            # Embedding API (text-embedding-3-small)
groq>=0.4.0              # LLM extraction (Llama 3 via Groq)

# Content Processing
pymupdf>=1.24.0          # PDF extraction
pytesseract>=0.3.10      # OCR
Pillow>=10.0.0           # Image processing
trafilatura>=1.8.0       # Web page extraction
tree-sitter>=0.22.0      # AST-aware code chunking
markdown-it-py>=3.0.0    # Markdown parsing

# Utilities
watchdog>=4.0.0          # File system monitoring
pydantic>=2.0.0          # Data validation
aiohttp>=3.9.0           # Async HTTP
rich>=13.0.0             # CLI output

# Optional
openai-whisper>=20231117 # Video transcription (pesante, opzionale)
```

Struttura Directory del Progetto

```
shard_memory/
├── __init__.py
├── client.py              # ShardMemory API principale
├── config.py              # Configurazione (API keys, paths, etc.)
│
├── ingest/
│   ├── __init__.py
│   ├── pipeline.py        # Pipeline di ingestion orchestrator
│   ├── chunkers/
│   │   ├── semantic.py    # Document chunking
│   │   ├── ast_chunker.py # Code chunking
│   │   ├── heading.py     # Markdown chunking
│   │   └── sliding.py     # Fallback chunking
│   └── extractors/
│       ├── pdf.py
│       ├── image.py
│       ├── web.py
│       ├── video.py
│       └── code.py
│
```

```
├── memory/
│   ├── __init__.py
│   ├── extractor.py        # Memory extraction via LLM
│   ├── contradiction.py    # Contradiction resolution
│   ├── derivation.py       # Insight derivation
│   └── forgetting.py       # Automatic forgetting & GC
│
├── graph/
│   ├── __init__.py
│   ├── sqlite_store.py     # SQLite operations
│   ├── vector_store.py     # ChromaDB operations
│   └── models.py           # Pydantic models (Memory, Document, etc.)
│
├── query/
│   ├── __init__.py
│   ├── search.py           # Hybrid search engine
│   ├── rerank.py           # LLM-based reranking
│   └── rewrite.py          # Query rewriting
│
├── profile/
│   ├── __init__.py
│   └── engine.py           # Profile generation & caching
│
├── connectors/
│   ├── __init__.py
│   ├── base.py             # BaseConnector ABC
│   ├── filesystem.py       # File system watcher
│   ├── session_log.py      # SHARD SSJ log importer
│   └── git.py              # Git repo sync
│
└── utils/
    ├── embeddings.py       # Embedding API wrapper
    ├── llm.py              # LLM API wrapper (Groq)
    └── logging.py          # Structured logging
```

Piano di Implementazione (Fasi)

Fase 1: Foundation (1-2 settimane)

- [ ] Setup progetto, config, models Pydantic
- [ ] SQLite schema + ChromaDB setup
- [ ] Embedding wrapper (OpenAI text-embedding-3-small)
- [ ] LLM wrapper (Groq Llama 3)
- [ ] `client.add()` base (testo raw → chunk → embed → store)

- [ ] `client.search()` base (vector search su chunks)

Fase 2: Memory Engine (2-3 settimane)

- [ ] Memory extraction via LLM prompt
- [ ] Contradiction resolution
- [ ] Memory relationships (UPDATES, EXTENDS, DERIVES)
- [ ] `is_latest` tracking
- [ ] Automatic forgetting + garbage collection
- [ ] Episode confidence decay

Fase 3: Profile & Hybrid Search (1-2 settimane)

- [ ] Profile engine (static + dynamic facts)
- [ ] Profile caching
- [ ] `client.profile()` endpoint
- [ ] Hybrid search mode (RAG + Memory)
- [ ] Query rewriting
- [ ] LLM-based reranking

Fase 4: Content Processing (2 settimane)

- [ ] PDF extractor
- [ ] Image OCR
- [ ] AST-aware code chunking
- [ ] Markdown heading chunker
- [ ] Web page extractor
- [ ] Conversation parser

Fase 5: Connectors & SHARD Integration (1-2 settimane)

- [ ] SessionLog connector (importa log SSJ)
- [ ] FileSystem watcher
- [ ] Git connector
- [ ] Integrazione nel loop principale SHARD
- [ ] Hook nel SHARD.STUDY pipeline

Fase 6: Optimization (ongoing)

- [ ] Benchmark con MemoryBench
- [ ] Tuning prompt di estrazione
- [ ] Caching embedding per ridurre API calls
- [ ] Batch processing per documenti grandi
- [ ] Dashboard CLI con Rich

Costi API Stimati (Geekom A5 + API Esterne)

Operazione	Provider	Costo Stimato
Embedding (text-embedding-3-small)	OpenAI	~$0.02 / 1M tokens
Memory extraction (Llama 3 70B)	Groq	Free tier / ~$0.59/1M tokens
Contradiction check	Groq	Incluso sopra
Profile generation	Groq	Incluso sopra
Query rewrite	Groq	Incluso sopra
Storage (SQLite + ChromaDB)	**Locale**	**$0**
Processing (chunking, OCR)	**Locale**	**$0**

Stima mensile per uso moderato: $2-5/mese (mainly embedding costs)

Note Architetturali

Perché SQLite + ChromaDB (non Postgres)

- Il Geekom A5 non ha bisogno di Postgres per questo workload
- SQLite è zero-config, embedded, velocissimo per single-writer
- ChromaDB è il vector store più leggero che gira in-process
- Se scala → migra a Postgres + pgvector senza cambiare l'API

Perché Groq per Extraction (non OpenAI)

- Groq ha free tier generoso per Llama 3
- Latenza bassissima (~100ms per extraction)
- SHARD già usa Groq nel suo stack
- Fallback su OpenAI se Groq down

Trade-off vs Supermemory Hosted

	SHARD.MEMORY (self-hosted)	Supermemory (SaaS)
Latenza search	~20-50ms (locale)	~50-100ms (rete)
Costo	~$3/mese (embedding)	Pricing TBD

	SHARD.MEMORY (self-hosted)	**Supermemory (SaaS)**
Privacy	100% locale	Dati su loro server
Connettori	Custom per SHARD	Google/Gmail/Notion
Manutenzione	Tu	Loro
Benchmark	Da verificare	#1 su tutti